MATRIXPOLICY FOR ROLE-AWARE OPTIMIZATION OF RATIONAL TRANSFORMER NONLINEARITIES

Anonymous authors
Paper under double-blind review

ABSTRACT

A useful LLM pretraining recipe moves the loss-time frontier by reaching the same validation loss with fewer tokens or less wall-clock time. We target this objective through the Transformer nonlinear sublayer, treating it as an optimizer-visible interface. MatrixPolicy with Rational Latent Basis (RLB) replaces a fixed scalar response with grouped rational responses that expose local gain, coefficient-activity, and matrix-balance statistics to a role-aware update. MatrixPolicy uses these signals to update only the sublayer input and output matrices, while rational coefficients and the rest of the Transformer remain on AdamW. Across DCLM, FineWeb-Edu, FineWeb, Dolma-sample, and C4 with a fixed 12-layer, width-768 decoder, MatrixPolicy reaches matched validation-loss targets with fewer tokens and lower measured arrival time than SiLU controls under the same protocol. It saves 19.5–22.8% target-arrival tokens versus SiLU with AdamW and 29.2–34.0% versus SiLU with Muon at 100M tokens, and 22.3–26.4% and 6.7–9.8%, respectively, at 300M tokens. [todo: Insert the larger-scale training result here.]

1 INTRODUCTION

Recent LLM optimizer papers frame practical efficiency through loss-time and compute-time frontiers, asking which training recipe reaches a target loss with fewer tokens, FLOPs, or wall-clock time (Shah et al., 2025; Liu et al., 2025; Semenov et al., 2025). This criterion also exposes an activation-design question. In a modern decoder block, the nonlinear sublayer is a learned matrix–nonlinearity–matrix map. The activation sets local gain and curvature, while the adjacent matrices choose and recombine the coordinates on which that response acts. Standard training pipelines leave this local structure implicit to the optimizer. Activation coefficients and adjacent matrices are updated as ordinary parameters, so matrix updates do not receive the gain, coefficient-activity, or role information generated inside the sublayer.

We make this interface concrete with Rational Latent Basis (RLB) and MatrixPolicy. RLB maps residual states into grouped latent coordinates, normalizes each group by its RMS radius, and applies a trainable real-pole-free P5/Q4 rational response. The resulting response-gain, coefficient-activity, matrix-pressure, and pair-balance signals are detached from the forward graph and passed to MatrixPolicy. MatrixPolicy uses them to update only the RLB input and output matrices; attention, embeddings, normalization parameters, rational coefficients, and ordinary Transformer tensors remain on AdamW. This local division lets the nonlinear sublayer expose the quantities it creates without changing the optimizer used for the rest of the model.

We evaluate target arrival by measuring the tokens and wall-clock time required to reach a fixed validation-loss target under the same model, corpus, and protocol. In the completed study, the same 12-layer, width-768 causal Transformer is trained on DCLM, FineWeb-Edu, FineWeb, Dolma-sample, and C4. The experiments include RLB-only controls and matched SiLU with AdamW and SiLU with Muon baselines under the same model, data, and measurement protocol. MatrixPolicy reaches shared validation-loss targets earlier at both 100M and 300M token budgets. [todo: Insert the larger-scale training result here.] The contribution is an activation design that exposes optimizer-visible matrix roles, a localized role-aware optimizer for the exposed matrix pair, and matched evidence that this interface reduces target-arrival tokens and wall-clock time across five pretraining corpora.

2 Related Work

Activation design for Transformer nonlinear sublayers has moved from fixed pointwise curves to gated matrix sublayers. Early Transformer-style sublayers used fixed pointwise responses such as GELU or Swish/SiLU (Vaswani et al., 2017; Hendrycks & Gimpel, 2016; Ramachandran et al., 2017). GLU introduced multiplicative feature selection for language modeling (Dauphin et al., 2017), GLU variants such as SwiGLU improved Transformer sublayers in compute-matched settings (Shazeer, 2020), and large decoder models adopted SwiGLU together with pre-normalized Transformer components (Chowdhery et al., 2022; Touvron et al., 2023). These works identify the nonlinear block as the natural comparison point for RLB because the response is coupled to the matrices that prepare and mix it. RLB keeps that sublayer location while replacing the fixed curve with grouped trainable rational responses.

Choosing a trainable rational response builds on earlier evidence that low-degree quotients can serve as learnable nonlinearities. Padé Activation Units learned such quotients end to end as activation functions (Molina et al., 2020), rational neural networks gave approximation efficiency results for smooth-function classes (Boullé et al., 2020), and RAFT evaluated rational activation functions inside Transformer pretraining and fine-tuning (Fang et al., 2023). Recent theory strengthens the motivation by proving expressivity advantages for trainable low-degree rational activations over a broad set of modern fixed activations, including gated and Transformer-style nonlinearities (Tang & Townsend, 2026). RLB therefore follows the rational-activation lineage; MatrixPolicy tests the additional effect of making the surrounding matrices optimizer-visible.

Making the surrounding matrices optimizer-visible connects RLB to a fast-moving LLM optimizer literature. Adam and AdamW remain central baselines (Kingma & Ba, 2015; Loshchilov & Hutter, 2019), while state-efficient or diagonal adaptive methods such as Adafactor and CAME reduce optimizer memory or alter adaptive-state estimates (Shazeer & Stern, 2018; Luo et al., 2023). Other recent methods change the update rule itself. Lion, Schedule-Free methods, and cautious optimizers alter momentum, sign, or schedule behavior (Chen et al., 2023; Defazio et al., 2024; Liang et al., 2026), while Sophia, GaLore, and Adam-mini change curvature, low-rank, or per-block learning-rate structure (Liu et al., 2024; Zhao et al., 2024; Zhang et al., 2025). Practical-efficiency studies evaluate optimizers by the tokens, compute, or time needed to reach matched losses (Shah et al., 2025; Schlotthauer et al., 2025; Semenov et al., 2025; Wen et al., 2026). The matrix restriction in MatrixPolicy connects most directly to matrix-aware optimization. Shampoo uses tensor preconditioners (Gupta et al., 2018), SOAP stabilizes Shampoo-style preconditioning by running Adam in the preconditioner eigenbasis (Vyas et al., 2025), and Muon uses orthogonalized matrix updates for LLM training (Liu et al., 2025). Recent optimizer comparisons emphasize fair tuning, model-scale dependence, and setting-dependent tradeoffs (Zhao et al., 2025; Wen et al., 2026). MatrixPolicy follows this matrix-aware direction, with updates restricted to the RLB input/output matrix pair and conditioned on activation-derived sublayer statistics.

3 Method

RLB and MatrixPolicy are designed as a single interface. RLB defines the nonlinear sublayer and the detached summaries it exposes; MatrixPolicy consumes those summaries to update the two adjacent matrices. Rational coefficients and ordinary Transformer parameters remain on the AdamW path in the evaluated configuration.

Throughout the method we use

$$\mathrm{clip}(x, a, b) = \min\{\max\{x, a\}, b\}, \tag{1}$$

$$\mathrm{smoothstep}(a, b, x) = 3\omega^2 - 2\omega^3, \qquad \omega = \mathrm{clip}\left(\frac{x-a}{b-a}, 0, 1\right). \tag{2}$$

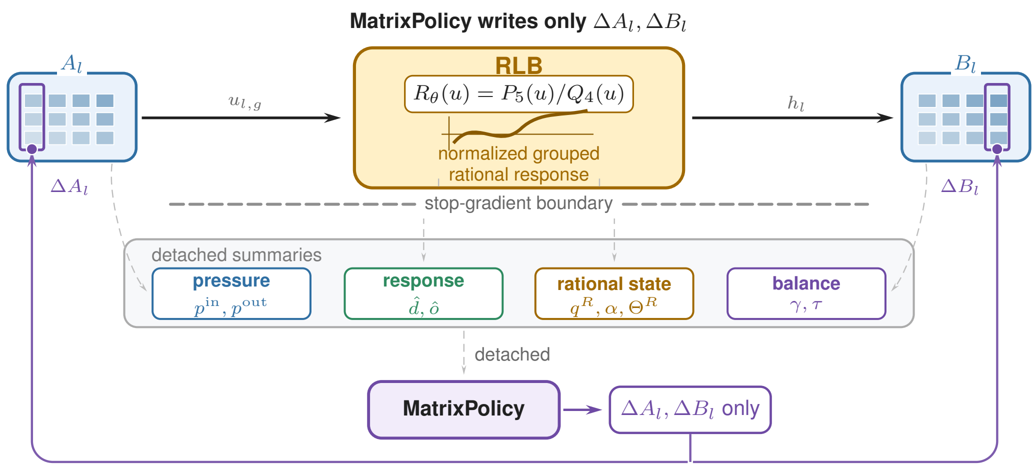

Figure 1: Activation–optimizer interface for RLB. In layer l, A_l maps the sublayer input into normalized rational groups and B_l recombines the rational features $h_l = \text{concat}_{g=1}^{G} h_{l,g}$. Forward RLB statistics, gradient pressure/activity EMAs, and pair-balance measurements cross a stop-gradient boundary as policy inputs. MatrixPolicy reads these summaries and writes only ΔA_l and ΔB_l; rational coefficients are AdamW-trained and policy-observed, not MatrixPolicy update targets.

3.1 Rational Latent Basis

For layer $l \in \{1, \ldots, L\}$, let $x_l \in \mathbb{R}^d$ be the input to the nonlinear sublayer. RLB uses an input matrix $A_l \in \mathbb{R}^{H \times d}$ and an output matrix $B_l \in \mathbb{R}^{d \times H}$ with latent width $H = Gm$. Let $A_{l,g} \in \mathbb{R}^{m \times d}$ denote the row block of A_l, and let $B_{l,g} \in \mathbb{R}^{d \times m}$ denote the matching column block of B_l. In the implementation, A_l and B_l are the bias-free `in_proj.weight` and `out_proj.weight` of the nonlinear sublayer. The evaluated runs use an RMS floor $\epsilon_{\text{rms}} = 10^{-6}$. Each group has a trainable P5/Q4 rational response

$$P_{l,g}(u) = \sum_{i=0}^{5} a_{l,g,i} u^i, \tag{3}$$

$$Q_{l,g}(u) = 1 + |b_{l,g,1}|\,|u| + |b_{l,g,2}|\,u^2 + |b_{l,g,3}|\,|u|^3 + |b_{l,g,4}|\,u^4, \tag{4}$$

$$R_{l,g}(u) = P_{l,g}(u)/Q_{l,g}(u). \tag{5}$$

Define the preactivation, RMS radius, normalized coordinate, and grouped rational feature by

$$z_{l,g}(x_l) = A_{l,g} x_l, \qquad \rho_{l,g}(x_l) = \sqrt{m^{-1}\|z_{l,g}(x_l)\|_2^2 + \epsilon_{\text{rms}}}, \tag{6}$$

$$u_{l,g}(x_l) = \frac{z_{l,g}(x_l)}{\rho_{l,g}(x_l)}, \qquad h_{l,g}(x_l) = \rho_{l,g}(x_l) R_{l,g}(u_{l,g}(x_l)), \qquad h_l = \text{concat}_{g=1}^{G} h_{l,g}. \tag{7}$$

The RLB forward map is the single block expression

$$y_l = B_l h_l = \sum_{g=1}^{G} B_{l,g} \rho_{l,g}(x_l) R_{l,g}\left(\frac{A_{l,g} x_l}{\rho_{l,g}(x_l)}\right). \tag{8}$$

The scalar functions $P_{l,g}$, $Q_{l,g}$, and $R_{l,g}$ are applied coordinatewise when their input is a vector. Since $Q_{l,g}(u) \geq 1$ for all real u, the implemented response has no real pole. The response is piecewise rational because of the absolute-value terms; derivative statistics use the autodiff convention $\text{sign}(0) = 0$ and equations involving $R'_{l,g}$ are interpreted almost everywhere. Numerator and denominator coefficients are initialized by a SiLU fit on $[-5, 5]$ and are then trained by AdamW. The evaluated model uses only these groupwise P5/Q4 rational responses.

The forward map in equation 8 separates the normalized coordinate at which the rational response is evaluated from the group radius returned to the residual stream. This radius-factor form exposes the positive homogeneity behind the matrix-pair coordinate formalized in Appendix B. The floor $\epsilon_{\mathrm{rms}} > 0$ is an additive squared-RMS stabilizer. For fixed curves $R_l = \{R_{l,g}\}_{g=1}^{G}$, write this block map as $f_l(x; A_l, B_l, R_l)$. For any fixed input x such that $A_{l,g}x \neq 0$ for every group, if $\epsilon_{\mathrm{rms}} = 0$, then for any $\lambda \in \mathbb{R}^G_{>0}$ and $D_l(\lambda) = \mathrm{diag}(\lambda_1 I_m, \ldots, \lambda_G I_m)$,

$$f_l(x; A_l, B_l, R_l) = f_l(x; D_l(\lambda)A_l, B_l D_l(\lambda)^{-1}, R_l). \tag{9}$$

This positive matrix-pair rescaling is exact only in the unfloored map. In the actual training dynamics with the stabilizing RMS floor and optimizer state, MatrixPolicy uses the same direction as a bounded balance coordinate.

3.2 MatrixPolicy

MatrixPolicy acts only on the RLB matrix pair (A_l, B_l). In the evaluated optimizer, all policy inputs are detached summaries. Its contract has three operations: group-gradient gating, a role/depth AdamW branch with an early Muon matrix substep, and scheduled reciprocal rescaling of the same positive matrix pair. The forward hook stores cached per-group response and derivative RMS summaries from sampled normalized coordinates. After backpropagation and global gradient clipping, the wrapper reads the optimizer-visible, pre-group-policy gradients on A_l, B_l, and the rational coefficients, updates detached EMA memories, and shares those memories with the matrix optimizer. The rational coefficients are observed for policy signals but are not MatrixPolicy update targets; in the evaluated runs they remain in the AdamW no-decay parameter group. The resulting summaries never change the forward computation directly. They scale matrix gradients, modulate matrix-step learning rates and Muon activation, and choose a bounded representative of the same RLB matrix pair after the matrix step.

The quantities below follow the implementation order. Gradient-derived statistics are computed before parameter updates, while the pair-rescaling ratio is measured after the AdamW/Muon matrix branch. For notational convenience set $M_{l,\mathrm{in}} = A_l$ and $M_{l,\mathrm{out}} = B_l$; the roles $\sigma \in \{\mathrm{in}, \mathrm{out}\}$ refer to these two matrices. For any tensor V, define

$$\mathrm{rms}(V) = \sqrt{\mathrm{mean}(V^2)}, \qquad \mathrm{rms}_\epsilon(V) = \sqrt{\mathrm{mean}(V^2) + \epsilon}. \tag{10}$$

The evaluated implementation uses $\epsilon_{\mathrm{live}} = 10^{-6}$ for cached forward gains and $\epsilon_p = \epsilon_{\mathrm{probe}} = 10^{-8}$ for pressure, matrix-ratio, and fixed-grid probe floors. The floors make the measurements positive and numerically stable for finite tensors; MatrixPolicy obtains bounded update controls through the clipping operations below. Let $\Theta^R_{l,g}$ collect the trainable rational-response coefficient blocks present in group (l, g); for the evaluated groupwise P5/Q4 response these are the numerator and denominator coefficient tensors. The count $|\Theta^R_{l,g}|$ below is a block count, and each mean is over scalar entries in one coefficient block. With optimizer-visible gradients, the instantaneous detached measurements are the input-matrix relative gradient p^{in}, the output-matrix relative gradient p^{out}, and the raw rational-coefficient gradient activity p^R:

$$p^{\mathrm{in}}_{l,g} = \frac{\mathrm{rms}_{\epsilon_p}(\nabla A_{l,g})}{\mathrm{rms}_{\epsilon_p}(A_{l,g})}, \qquad p^{\mathrm{out}}_{l,g} = \frac{\mathrm{rms}_{\epsilon_p}(\nabla B_{l,g})}{\mathrm{rms}_{\epsilon_p}(B_{l,g})}, \tag{11}$$

$$p^R_{l,g} = \left[\frac{1}{|\Theta^R_{l,g}|} \sum_{\theta \in \Theta^R_{l,g}} \mathrm{mean}((\nabla\theta)^2) + \epsilon_p \right]^{1/2}. \tag{12}$$

The same numerical floor is used in $p^R_{l,g}$, where the measured quantity is raw coefficient-gradient energy rather than a gradient-to-weight ratio. The displayed p values are evaluated at the current optimizer step; in the EMA recurrence, $p^r_{l,g,t}$ denotes that step-t value for role $r \in \{\mathrm{in}, \mathrm{out}, R\}$. MatrixPolicy does not use the instantaneous values directly. It stores detached EMA memories q^{in}, q^{out}, and q^R,

$$q^r_{l,g,t} = 0.95\, q^r_{l,g,t-1} + 0.05\, p^r_{l,g,t}, \qquad r \in \{\mathrm{in}, \mathrm{out}, R\}, \tag{13}$$

initialized from the first observed post-global-clipping values. The symbol q therefore denotes the policy memory of a detached measurement, not a rational denominator. The two contrasts used by later rules are

$$\pi_{l,g} = \log q^{\mathrm{in}}_{l,g} - \log q^{\mathrm{out}}_{l,g}, \qquad \alpha_{l,g} = \log q^R_{l,g} - \frac{1}{2}\left(\log q^{\mathrm{in}}_{l,g} + \log q^{\mathrm{out}}_{l,g}\right). \tag{14}$$

Each matrix pressure is an unsigned floored relative RMS-gradient proxy, so $\pi_{l,g}$ is signed only because it compares the input and output proxy memories. MatrixPolicy uses clipped transforms of $\pi_{l,g}$ as pressure controls for redistribution, Muon attenuation, and pair balancing. The score $\alpha_{l,g}$ compares rational-coefficient activity with the geometric mean of the adjacent matrix-pressure memories; its clipped or ReLU-transformed versions are used for damping and rescale speed. It is an implementation contrast, not a scale-invariant physical ratio. Appendix C.1 gives the local gradient geometry behind these proxy choices. The step subscript is dropped below when the time is clear.

The activation also provides cached sampled curve gains. The RLB forward statistics hook periodically subsamples activation inputs, forms normalized coordinates, and stores per-group response and derivative RMS summaries. These are cached forward summaries rather than differentiable live inputs to the optimizer. MatrixPolicy reads the floored summaries

$$\hat{d}^{\text{live}}_{l,g} = \sqrt{\operatorname{mean}\Big(R'_{l,g}(u_{l,g})^2\Big) + \epsilon_{\text{live}}}, \qquad \hat{o}^{\text{live}}_{l,g} = \sqrt{\operatorname{mean}(R_{l,g}(u_{l,g})^2) + \epsilon_{\text{live}}}. \tag{15}$$

The mean is over sampled scalar normalized coordinates for group (l, g) across the most recent refreshed forward-statistics sample, sequence positions, and the m group coordinates. The derivative gain is the input-role proxy; the response gain is the recombination-role proxy. The latter excludes the group radius $\rho_{l,g}(x_l)$, so neither gain is a full feature scale or block-Jacobian preconditioner.

The detached summaries have distinct consumers. The group-gradient gate reads the cached live gains, the pressure contrast $\pi_{l,g}$, and the coefficient-activity contrast $\alpha_{l,g}$, then rescales only the matrix gradients. The matrix branch uses a role/depth AdamW learning-rate multiplier; in the evaluated configuration, pressure and activity enter this branch through the early Muon attenuation factor. The surrounding wrapper then performs a reciprocal pair rescale using fixed-grid probe gains, $\pi_{l,g}$, $\alpha_{l,g}$, and the post-branch matrix log-ratio $\gamma_{l,g}$. No summary tensor enters the forward graph, and no summary directly updates the rational coefficients.

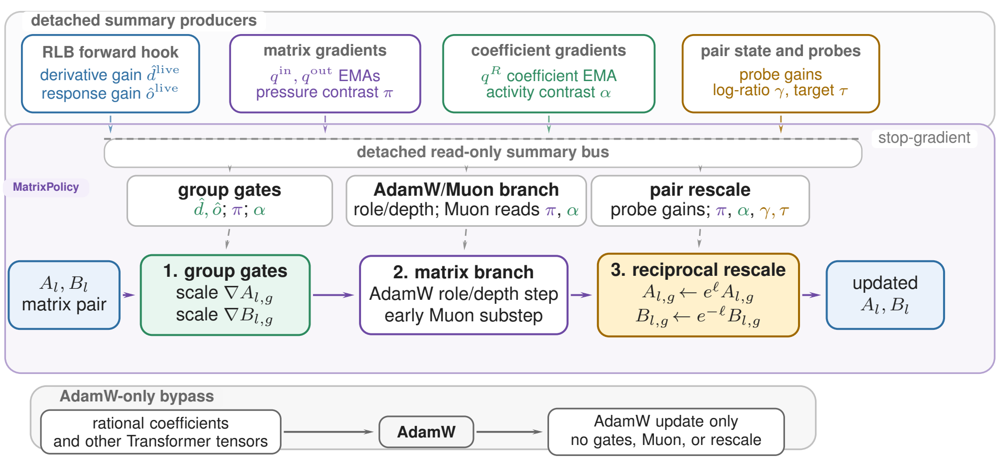

Figure 2: Detached-summary route for the evaluated MatrixPolicy system. The top band shows where each summary family is produced, the read-only summary bus carries those detached signals to the MatrixPolicy consumers, and the bottom lane separates AdamW-only tensors from the matrix-pair update path.

Let t be the one-based internal optimizer step after the wrapper increments its counter, T the planned number of training steps, and $\xi_t = t/T$. A smooth gate $\phi_t = \operatorname{smoothstep}(0.02, 0.30, \xi_t)$ turns on the group-gradient policy. The evaluated configuration uses gain, pressure, and activity strengths 0.20, 0.10, and 0.20, and clips group multipliers to $[0.75, 1.35]$. For positive group values v_j, define

$\mathrm{geomean}_{j=1}^{G}(v_j) = \exp\Big(G^{-1}\sum_{j=1}^{G}\log v_j\Big)$. The complete group-gradient rule is

$$\begin{aligned}
\tilde{\pi}_{l,g} &= \mathrm{clip}(\pi_{l,g}, -1.50, 1.50), & c_{l,g}^{\mathrm{activity}} &= \exp\left[-0.20\phi_t\,\mathrm{ReLU}\left(\frac{\alpha_{l,g}-0.05}{0.45}\right)\right],\\
c_{l,g,\mathrm{in}}^{\mathrm{gain}} &= \left(\frac{\mathrm{geomean}_{j=1}^{G}\,\hat{d}_{l,j}^{\mathrm{live}}}{\hat{d}_{l,g}^{\mathrm{live}}}\right)^{0.20\phi_t}, & c_{l,g,\mathrm{out}}^{\mathrm{gain}} &= \left(\frac{\mathrm{geomean}_{j=1}^{G}\,\hat{o}_{l,j}^{\mathrm{live}}}{\hat{o}_{l,g}^{\mathrm{live}}}\right)^{0.20\phi_t},\\
c_{l,g,\mathrm{in}}^{\mathrm{pressure}} &= \exp(-0.10\phi_t\tilde{\pi}_{l,g}), & c_{l,g,\mathrm{out}}^{\mathrm{pressure}} &= \exp(+0.10\phi_t\tilde{\pi}_{l,g}),\\
c_{l,g,\sigma}^{\mathrm{raw}} &= c_{l,g,\sigma}^{\mathrm{gain}}c_{l,g,\sigma}^{\mathrm{pressure}}c_{l,g}^{\mathrm{activity}}, & c_{l,g,\sigma} &= \mathrm{clip}\left(\frac{c_{l,g,\sigma}^{\mathrm{raw}}}{\mathrm{geomean}_{j=1}^{G}\,c_{l,j,\sigma}^{\mathrm{raw}}}, 0.75, 1.35\right).
\end{aligned} \tag{16}$$

Thus live derivative gain controls the input role, live response gain controls the output role, relative-gradient pressure acts with opposite signs on the two roles, and coefficient activity downweights groups whose coefficient-gradient activity is large relative to the adjacent matrix proxies. A positive $\pi_{l,g}$ means the input-side relative-gradient EMA is larger than the output-side EMA; the gate therefore reduces input-role step scale and increases output-role step scale. The geometric recentering in equation 16 redistributes update scale across groups without changing the role-wise mean log multiplier before clipping.

The scaled matrix gradients are

$$\widetilde{\nabla}A_{l,g} = c_{l,g,\mathrm{in}}\nabla A_{l,g}, \qquad \widetilde{\nabla}B_{l,g} = c_{l,g,\mathrm{out}}\nabla B_{l,g}. \tag{17}$$

Let $\widetilde{\nabla}M_{l,\mathrm{in}}$ be the row-block concatenation of $\{\widetilde{\nabla}A_{l,g}\}_{g=1}^{G}$, and let $\widetilde{\nabla}M_{l,\mathrm{out}}$ be the column-block concatenation of $\{\widetilde{\nabla}B_{l,g}\}_{g=1}^{G}$. After this scaling, the RLB matrices are updated by a role/depth AdamW branch and a transient Muon branch. Let $\delta_l = (l-1)/(L-1)$ for $L > 1$ and $\delta_l = 1/2$ otherwise. The evaluated branch schedules are

$$\begin{aligned}
\varrho_{\mathrm{in}}(l) &= \mathrm{clip}(1 - 0.50(\delta_l - 0.5), 0.55, 1.40),\\
\varrho_{\mathrm{out}}(l) &= \mathrm{clip}(1 + 1.00(\delta_l - 0.5), 0.55, 1.40),\\
s_{l,\sigma}^{\mathrm{Adam}} &= \mathrm{clip}(3.0\max\{0.10, 1 + 1.20(\varrho_\sigma(l) - 1)\}, 0.40, 4.0)\,,\\
\Psi_{l,t} &= \mathrm{clip}\left(G^{-1}\sum_{g=1}^{G}|\pi_{l,g}|, 0, 1.50\right),\\
\Omega_{l,t} &= \mathrm{ReLU}\left(\frac{G^{-1}\sum_{g=1}^{G}\alpha_{l,g} - 0.05}{0.45}\right),\\
\psi_{l,t} &= \mathrm{clip}(\exp[-0.30\Psi_{l,t} - 0.65\Omega_{l,t}], 0.10, 1.15)\,,\\
\omega_t^{\mu} &= \mathrm{smoothstep}(0.02, 0.12, \xi_t)[1 - \mathrm{smoothstep}(0.20, 0.36, \xi_t)],\\
\mu_{l,\sigma,t} &= \mathrm{clip}(0.75\,\omega_t^{\mu}\varrho_\sigma(l)\psi_{l,t}, 0, 0.75)\,.
\end{aligned} \tag{18}$$

Here $s_{l,\sigma}^{\mathrm{Adam}}$ is the role/depth AdamW multiplier. The aggregate pressure $\Psi_{l,t}$ and coefficient activity $\Omega_{l,t}$ form the Muon attenuation $\psi_{l,t}$, so $\mu_{l,\sigma,t}$ is large only during the early Muon window and when the detached summaries do not indicate high imbalance or high coefficient-gradient activity. For $M_{l,\mathrm{in}} = A_l$ and $M_{l,\mathrm{out}} = B_l$, let η_t be the base learning rate. MatrixPolicy applies an AdamW matrix step followed by any active Muon substep. The gate $\mu_{l,\sigma,t}$ scales the two learning rates; it is not a convex mixing coefficient for a single update direction:

$$\begin{aligned}
\widetilde{M}_{l,\sigma}^{t+1} &= \mathrm{AdamW}_{\eta_t s_{l,\sigma}^{\mathrm{Adam}}(1-\mu_{l,\sigma,t})}(M_{l,\sigma}^{t}; \widetilde{\nabla}M_{l,\sigma}^{t}),\\
M_{l,\sigma}^{t+1} &= \mathrm{Muon}_{\eta_t\mu_{l,\sigma,t}}(\widetilde{M}_{l,\sigma}^{t+1}; \widetilde{\nabla}M_{l,\sigma}^{t}),
\end{aligned} \tag{19}$$

The matrix branch in equation 19 is operational: AdamW and Muon keep separate states, use their configured decay and momentum terms, and both consume the step-t gradient. Here Muon is the `torch.optim.Muon` update on two-dimensional RLB matrices with momentum 0.95, five

Newton–Schulz steps, and match-RMS learning-rate adjustment (Liu et al., 2025); it is inactive after all $\mu_{l,\sigma,t}$ decay to zero.

The surrounding on-policy wrapper's last MatrixPolicy-specific operation is a bounded positive rescaling of the two matrix blocks around each rational curve. It is evaluated after the AdamW/Muon matrix branch every five optimizer steps; fixed-grid probe gains are refreshed every ten optimizer steps and reused between refreshes. Let $\mathcal{U} = \{u_k\}_{k=1}^{129}$ be a uniform probe grid over $[-5, 5]$, and define $\mathrm{rms}_{\mathcal{U}}(\varphi) = \sqrt{|\mathcal{U}|^{-1} \sum_{u_k \in \mathcal{U}} \varphi(u_k)^2}$. Fixed-grid probe gains describe curve shape for pair balancing; live gains in equation 15 describe the latest cached sampled forward distributions for group-gradient redistribution. The first term of the target below equalizes derivative and response probe gains in the positive log coordinate derived in Appendix C.2; the pressure term lets recent role-imbalance shift that target within a bounded range. The wrapper computes

$$\begin{aligned}
\hat{d}_{l,g}^{\mathrm{probe}} &= \sqrt{\mathrm{rms}_{\mathcal{U}}(R'_{l,g})^2 + \epsilon_{\mathrm{probe}}},\\
\hat{o}_{l,g}^{\mathrm{probe}} &= \sqrt{\mathrm{rms}_{\mathcal{U}}(R_{l,g})^2 + \epsilon_{\mathrm{probe}}},\\
\gamma_{l,g} &= \log \mathrm{rms}_{\epsilon_p}(A_{l,g}) - \log \mathrm{rms}_{\epsilon_p}(B_{l,g}),\\
\bar{\pi}_{l,g} &= \mathrm{clip}(\pi_{l,g}, -1.25, 1.25),\\
\tau_{l,g} &= \frac{1}{2}\left(\log \hat{d}_{l,g}^{\mathrm{probe}} - \log \hat{o}_{l,g}^{\mathrm{probe}}\right) + 0.25\,\bar{\pi}_{l,g},\\
\bar{\alpha}_{l,g} &= \mathrm{clip}(\alpha_{l,g}, -1.25, 1.25),\\
\chi_{l,g}^{\mathrm{rs}} &= \exp(0.10\,\bar{\alpha}_{l,g}),\\
s_{l,t} &= 0.50\ \mathrm{smoothstep}(0.03, 0.35, \xi_t)\,\mathrm{clip}(1 + 0.15(\delta_l - 0.5), 0.75, 1.25).
\end{aligned} \tag{20}$$

The clipped log step and reciprocal matrix update are

$$\begin{aligned}
\ell_{l,g,t} &= \mathrm{clip}\left(\frac{1}{2}[\tau_{l,g} - \gamma_{l,g}] s_{l,t} \chi_{l,g}^{\mathrm{rs}}, -0.030, 0.030\right),\\
A_{l,g} &\leftarrow e^{\ell_{l,g,t}} A_{l,g}, \qquad B_{l,g} \leftarrow e^{-\ell_{l,g,t}} B_{l,g}.
\end{aligned} \tag{21}$$

Here $\gamma_{l,g}$ is a floored matrix-RMS proxy. The first term of $\tau_{l,g}$ equalizes derivative and response probe gains in the positive log coordinate, the pressure term shifts the target boundedly, and χ^{rs} changes only the clipped rescale speed; larger $\alpha_{l,g}$ accelerates only this representative-balancing move, while earlier group gates and Muon penalties use the same activity contrast to reduce gradient-driven matrix motion. Since equation 20 is floored, equation 21 is exact only in the nonzero, unfloored idealization; near the floor it is a clipped balance step. Optimizer-state buffers are scaled covariantly: first-moment-like entries for $A_{l,g}$ use $e^{\ell_{l,g,t}}$, first-moment-like entries for $B_{l,g}$ use $e^{-\ell_{l,g,t}}$, and square-average entries use the corresponding squared scales.

4 Experiments

4.1 Fixed-Scale Target-Arrival Study

The fixed-scale study uses the same 12-layer, width-768 causal Transformer on five corpora: DCLM/DataComp-LM (Li et al., 2024), FineWeb-Edu and FineWeb (Penedo et al., 2024), Dolma-sample (Soldaini et al., 2024), and C4 (Raffel et al., 2020). The 100M-token budget uses 3050 optimizer steps and the 300M-token budget uses 9150 optimizer steps, with three seeds per method/dataset pair. The main comparison keeps the architecture and data fixed while changing the activation and optimizer used in the nonlinear sublayer: MatrixPolicy is compared with SiLU controls and with RLB-only controls, where RLB-only uses the same RLB activation but no MatrixPolicy updates. Shared optimizer, evaluation-cadence, and throughput-measurement details are reported in Appendix A.1; broader optimizer comparisons beyond AdamW and Muon are summarized in Table 2.

Table 1: Target-arrival efficiency for the fixed 12-layer, width-768 Transformer. RLB denotes the same rational latent-basis activation without MatrixPolicy. At the hardest validation-loss target reached by every listed method in all three seeds, MatrixPolicy is always fastest; each token-saving column reports MatrixPolicy's savings relative to the method named below it.

			MatrixPolicy token savings (%)				Time to target (min)				
Budget	Dataset	Target loss	vs. SiLU AdamW	vs. SiLU Muon	vs. RLB AdamW	vs. RLB Muon	Matrix Policy	SiLU AdamW	SiLU Muon	RLB AdamW	RLB Muon
100M	DCLM	4.55	20.7	32.4	20.7	34.3	12.8	20.8	26.0	14.3	19.0
	FineWeb-Edu	4.40	19.5	29.5	20.2	29.5	12.9	20.3	24.4	14.2	18.0
	FineWeb	4.60	22.8	32.1	21.5	32.1	12.6	16.4	20.2	16.2	20.1
	Dolma	4.60	22.0	34.0	22.7	34.9	12.8	17.6	22.0	17.3	22.2
	C4	4.60	20.7	29.2	19.3	28.7	11.6	14.3	17.3	14.9	18.8
300M	DCLM	4.10	23.3	8.3	24.0	7.0	39.6	53.5	45.9	54.7	45.6
	FineWeb-Edu	3.85	22.8	6.7	22.6	4.8	40.0	59.9	47.9	52.3	50.0
	FineWeb	4.10	23.1	7.1	22.4	5.3	40.8	61.2	46.4	55.6	46.6
	Dolma	3.95	22.3	9.8	22.3	6.5	44.4	49.3	52.3	50.5	47.4
	C4	4.00	26.4	8.5	25.3	6.5	41.8	64.7	55.5	59.4	51.9

Table 1 is the primary empirical result. It asks: for the hardest common validation-loss target reached by every listed method in all three seeds, what fraction of target-arrival tokens does MatrixPolicy save, and how many measured target-arrival minutes does each method require? Across all five datasets under the 100M-token budget, MatrixPolicy saves 19.5–22.8% of the SiLU with AdamW target-arrival tokens and 29.2–34.0% of the SiLU with Muon target-arrival tokens. Under the 300M-token budget, after longer training compresses endpoint gaps, MatrixPolicy still saves 22.3–26.4% versus SiLU with AdamW and 6.7–9.8% versus SiLU with Muon, with lower target-arrival time in every listed comparison.

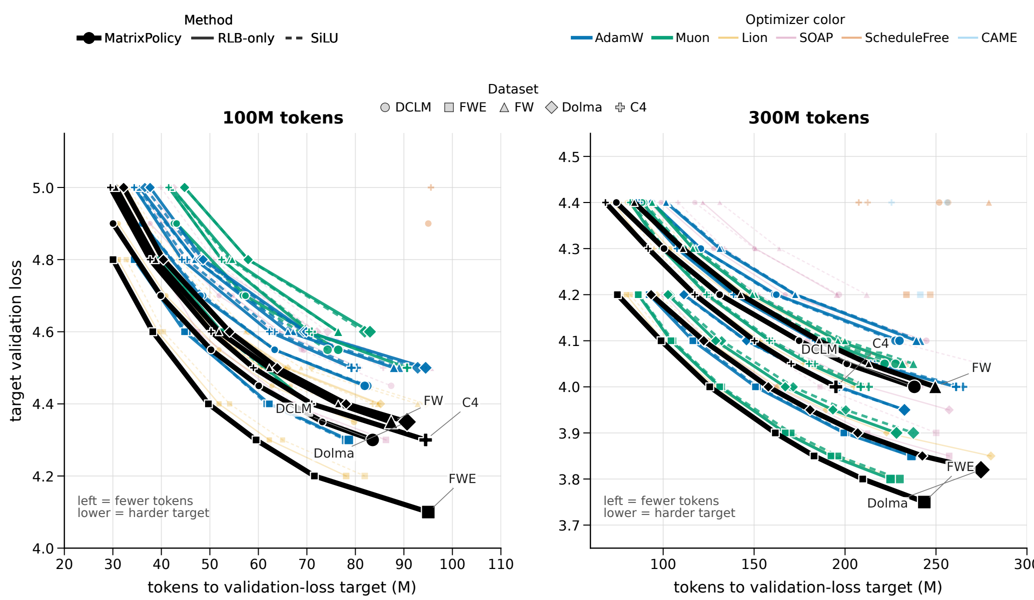

Figure 3: Target-arrival map for the completed fixed-scale study. Each point is the mean token count needed by one method to reach one validation-loss target on one dataset; connected points trace the same method and dataset from easier to harder targets. The horizontal coordinate is target-arrival tokens, and the vertical coordinate is the target validation loss, so curves further left reach the same loss with fewer tokens. The hard-target timing and token-saving percentages are quantified in Table 1.

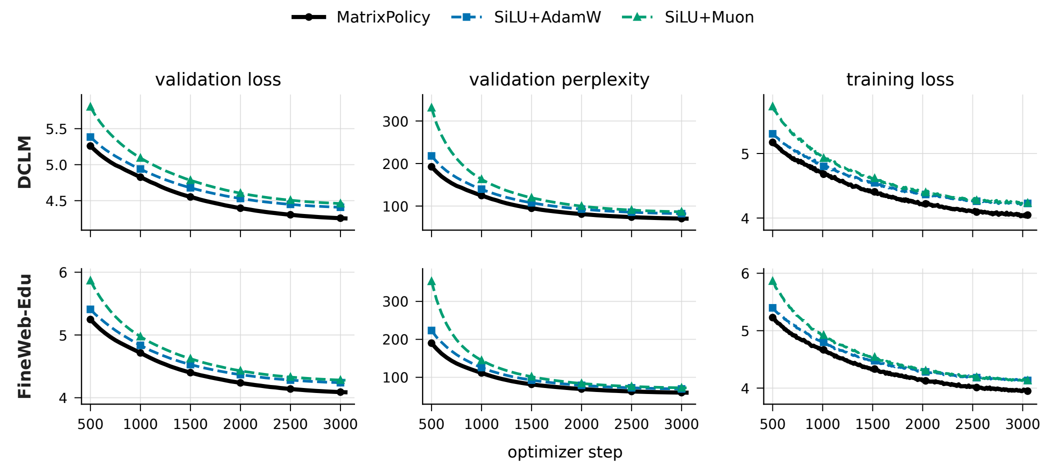

Figure 4: Representative 100M-token-budget training dynamics on DCLM and FineWeb-Edu. Rows show datasets and columns show validation loss, validation perplexity, and training loss. Lines are seed means over three runs and bands show one sample standard deviation. This figure uses the two standard SiLU baselines so that the early target-arrival effect in Table 1 remains visually legible.

The 100M-token-budget curves show why target arrival is the right main readout. MatrixPolicy separates early in validation loss and validation perplexity while keeping training loss competitive. A long fixed budget eventually compresses many activation/optimizer endpoints, especially under the 300M-token budget, so the paper emphasizes the number of tokens and minutes needed to reach the same useful validation-loss levels.

4.2 TODO: Larger-Scale Training Results

TODO: Insert the larger-scale training result here.

4.3 TODO: Component Ablations

TODO: Insert the component ablation study here.

5 Conclusion

RLB turns the nonlinear Transformer sublayer into an optimizer-visible interface by exposing grouped normalized coordinates, trainable groupwise rational responses, and an idealized positive matrix-pair relation used as a bounded balancing coordinate. MatrixPolicy uses that interface to update the RLB input and output matrices with role-aware statistics, while rational coefficients and non-RLB-matrix parameters use AdamW. The completed 100M-token and 300M-token runs show that MatrixPolicy reaches common validation losses with fewer tokens and lower measured target-arrival time than the listed SiLU and RLB controls. [todo: Complete this conclusion after the larger-scale and ablation results are added.]

References

Nicolas Boullé, Yuji Nakatsukasa, and Alex Townsend. Rational Neural Networks. In *Advances in Neural Information Processing Systems*, 2020. URL `https://proceedings.neurips.cc/paper/2020/hash/a3f390d88e4c41f2747bfa2f1b5f87db-Abstract.html`.

Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Lu, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic Discovery of Optimization Algorithms. In *Advances in Neural Information Processing Systems*,

2023. URL https://papers.neurips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, et al. PaLM: Scaling Language Modeling with Pathways. *arXiv preprint arXiv:2204.02311*, 2022. URL https://arxiv.org/abs/2204.02311.

Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. Language Modeling with Gated Convolutional Networks. In *Proceedings of the 34th International Conference on Machine Learning*, pp. 933–941, 2017. URL https://proceedings.mlr.press/v70/dauphin17a.html.

Aaron Defazio, Xingyu Yang, Ahmed Khaled, Konstantin Mishchenko, Harsh Mehta, and Ashok Cutkosky. The Road Less Scheduled. In *Advances in Neural Information Processing Systems*, 2024. URL https://neurips.cc/virtual/2024/poster/96925.

Haishuo Fang, Ji-Ung Lee, Nafise Sadat Moosavi, and Iryna Gurevych. Transformers with Learnable Activation Functions. In *Findings of the Association for Computational Linguistics: EACL 2023*, 2023. URL https://aclanthology.org/2023.findings-eacl.181/.

Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned Stochastic Tensor Optimization. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 1842–1850, 2018. URL https://proceedings.mlr.press/v80/gupta18a.html.

Dan Hendrycks and Kevin Gimpel. Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016. URL https://arxiv.org/abs/1606.08415.

Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*, 2015. URL https://mlanthology.org/iclr/2015/kingma2015iclr-adam/.

Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, et al. DataComp-LM: In Search of the Next Generation of Training Sets for Language Models. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-0455. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/19e4ea30dded58259665db375885e412-Abstract-Datasets_and_Benchmarks_Track.html.

Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious Optimizers: Improving Training with One Line of Code. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=zBPZeRjfgu.

Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/06960915ba8674c7a898ec0b472b80ff-Abstract-Conference.html.

Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is Scalable for LLM Training. *arXiv preprint arXiv:2502.16982*, 2025. URL https://arxiv.org/abs/2502.16982.

Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=Bkg6RiCqY7.

Yang Luo, Xiaozhe Ren, Zangwei Zheng, Zhuo Jiang, Xin Jiang, and Yang You. CAME: Confidence-guided Adaptive Memory Efficient Optimization. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pp. 4442–4453, 2023. doi: 10.18653/v1/2023.acl-long.243. URL https://aclanthology.org/2023.acl-long.243/.

Alejandro Molina, Patrick Schramowski, and Kristian Kersting. Padé Activation Units: End-to-End Learning of Flexible Activation Functions in Deep Networks. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=BJlBSkHtDS.

Guilherme Penedo, Hynek Kydlicek, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-0970. URL https://papers.nips.cc/paper/2024/hash/370df50ccfdf8bde18f8f9c2d9151bda-Abstract-Datasets_and_Benchmarks_Track.html.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL http://jmlr.org/papers/v21/20-074.html.

Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for Activation Functions. *arXiv preprint arXiv:1710.05941*, 2017. URL https://arxiv.org/abs/1710.05941.

Joel Schlotthauer, Christian Kroos, Chris Hinze, Viktor Hangya, Luzian Hahn, and Fabian Küch. Pre-Training LLMs on a Budget: A Comparison of Three Optimizers. *arXiv preprint arXiv:2507.08472*, 2025. URL https://arxiv.org/abs/2507.08472.

Andrei Semenov, Matteo Pagliardini, and Martin Jaggi. Benchmarking Optimizers for Large Language Model Pretraining. *arXiv preprint arXiv:2509.01440*, 2025. URL https://arxiv.org/abs/2509.01440.

Ishaan Shah, Anthony M. Polloreno, Karl Stratos, Philip Monk, Adarsh Chaluvaraju, Andrew Hojel, Andrew Ma, Anil Thomas, Ashish Tanwer, Darsh J. Shah, Khoi Nguyen, Kurt Smith, Michael Callahan, Michael Pust, Mohit Parmar, Peter Rushton, Platon Mazarakis, Ritvik Kapila, Saurabh Srivastava, Somanshu Singla, Tim Romanski, Yash Vanjani, and Ashish Vaswani. Practical Efficiency of Muon for Pretraining. *arXiv preprint arXiv:2505.02222*, 2025. URL https://arxiv.org/abs/2505.02222.

Noam Shazeer. GLU Variants Improve Transformer. *arXiv preprint arXiv:2002.05202*, 2020. URL https://arxiv.org/abs/2002.05202.

Noam Shazeer and Mitchell Stern. Adafactor: Adaptive Learning Rates with Sublinear Memory Cost. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 4596–4604, 2018. URL https://proceedings.mlr.press/v80/shazeer18a.html.

Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, et al. Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. *arXiv preprint arXiv:2402.00159*, 2024. URL https://arxiv.org/abs/2402.00159.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. *arXiv preprint arXiv:2104.09864*, 2021. URL https://arxiv.org/abs/2104.09864.

Maosen Tang and Alex Townsend. Rational Neural Networks have Expressivity Advantages. *arXiv preprint arXiv:2602.12390*, 2026. doi: 10.48550/arXiv.2602.12390. URL https://arxiv.org/abs/2602.12390.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, et al. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971*, 2023. URL https://arxiv.org/abs/2302.13971.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, 2017. URL https://papers.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

Nikhil Vyas, Depen Morwani, Rosie Zhao, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and Stabilizing Shampoo using Adam for Language Modeling. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/e988664070e9591f93fdcf605f7dc623-Abstract-Conference.html.

Kaiyue Wen, David Leo Wright Hall, Tengyu Ma, and Percy Liang. Fantastic Pretraining Optimizers and Where to Find Them. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=2J51qUZ0iG.

Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. *arXiv preprint arXiv:1910.07467*, 2019. URL https://arxiv.org/abs/1910.07467.

Yushun Zhang, Congliang Chen, Ziniu Li, Tian Ding, Chenwei Wu, Diederik P. Kingma, Yinyu Ye, Zhi-Quan Luo, and Ruoyu Sun. Adam-mini: Use Fewer Learning Rates To Gain More. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/45ae878717399e6f62d57c65f052cd46-Abstract-Conference.html.

Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection. In *International Conference on Machine Learning*, 2024. URL https://openreview.net/forum?id=hYHsrKDiX7.

Rosie Zhao, Depen Morwani, David Brandfonbrener, Nikhil Vyas, and Sham Kakade. Deconstructing What Makes a Good Optimizer for Language Models. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2407.07972.

A Experiment Appendix

This appendix gives the fixed-scale protocol behind Section 4.1, the full training curves for the five corpora, and the broader optimizer endpoint comparison used to contextualize the main SiLU and RLB controls.

A.1 Fixed-Scale Target-Arrival Study

A.1.1 Model and Data Pipeline

The fixed-scale experiments use a custom causal decoder Transformer. The block uses a LLaMA-style pre-normalized layout with RMSNorm before attention and before the nonlinear sublayer, bias-free query/key/value and output attention projections, rotary position embeddings, causal scaled-dot-product attention, residual attention and nonlinear-sublayer updates, a final RMSNorm, and a tied token-embedding/output head (Zhang & Sennrich, 2019; Su et al., 2021; Touvron et al., 2023). The fixed model has 12 layers, width 768, 12 heads of dimension 64, and context length 256.

The SiLU controls use a SwiGLU-like gated nonlinear sublayer with bias-free matrices $W_g, W_v \in \mathbb{R}^{768\times 2048}$ and $W_o \in \mathbb{R}^{2048\times 768}$, computing $(\mathrm{SiLU}(xW_g) \odot xW_v)W_o$. MatrixPolicy and RLB-only configurations use a matched single-branch matrix pair. An input matrix maps width 768 to 3072 latent coordinates, the coordinates are partitioned into groups and transformed by the real-pole-free P5/Q4 rational response, and an output matrix maps the 3072 coordinates back to width

768. With the default group size 256, this gives $G = 12$ groups of width $m = 256$; the RMS floor is $\epsilon_{\mathrm{rms}} = 10^{-6}$. This is the RLB configuration used by the main MatrixPolicy and RLB-control results.

Tokenization uses the GPT-2 tokenizer through the Hugging Face tokenizer interface, without adding tokenizer-specific special tokens; an EOS or pad token is appended between documents. Training samples random contiguous blocks of length 257 and forms next-token prediction pairs of length 256. The DCLM, FineWeb-Edu, FineWeb, Dolma-sample, and C4 runs use the text field specified in the manifests, with validation slices drawn from held-out token offsets of the same corpus stream.

A.1.2 Training Protocol

All runs use batch size 16, gradient accumulation 2, and therefore 32768 global tokens per optimizer step. The 100M-token budget consists of 3050 optimizer steps; the 300M-token budget consists of 9150 optimizer steps. Each method/dataset pair is run with seeds 1337, 2027, and 3407. Validation is performed every 50 optimizer steps, so token-to-target arrivals are quantized in increments of 1.64M tokens.

The MatrixPolicy rows in the main table use the RLB activation, whose active nonlinearity is the groupwise P5/Q4 response in equations 3–5. The evaluated configuration uses AdamW as the backbone optimizer and disables the separate rational-coefficient optimizer. It enables MatrixPolicy group redistribution with gain/pressure/activity strengths $0.20/0.10/0.20$, schedule window 0.02–0.30, and group multiplier clip $[0.75, 1.35]$. The RLB matrix pair then uses the role/depth-scaled AdamW branch and the early Muon substep from Section 3.2. The bounded RLB pair-rescale wrapper runs every five optimizer steps with covariant optimizer-state scaling. Rational coefficients, attention weights, embeddings, normalization parameters, and ordinary Transformer weights use AdamW in this configuration.

Target-arrival minutes in Table 1 are computed seed-wise from the first validation event that reaches the shared target and the same run's measured seconds per optimizer step. The per-step time is the training-script wall time recorded in the run summary divided by completed optimizer steps for that run; target arrivals are therefore measured with the same evaluation cadence and timing source across methods.

A.1.3 Supporting Fixed-Scale Curves

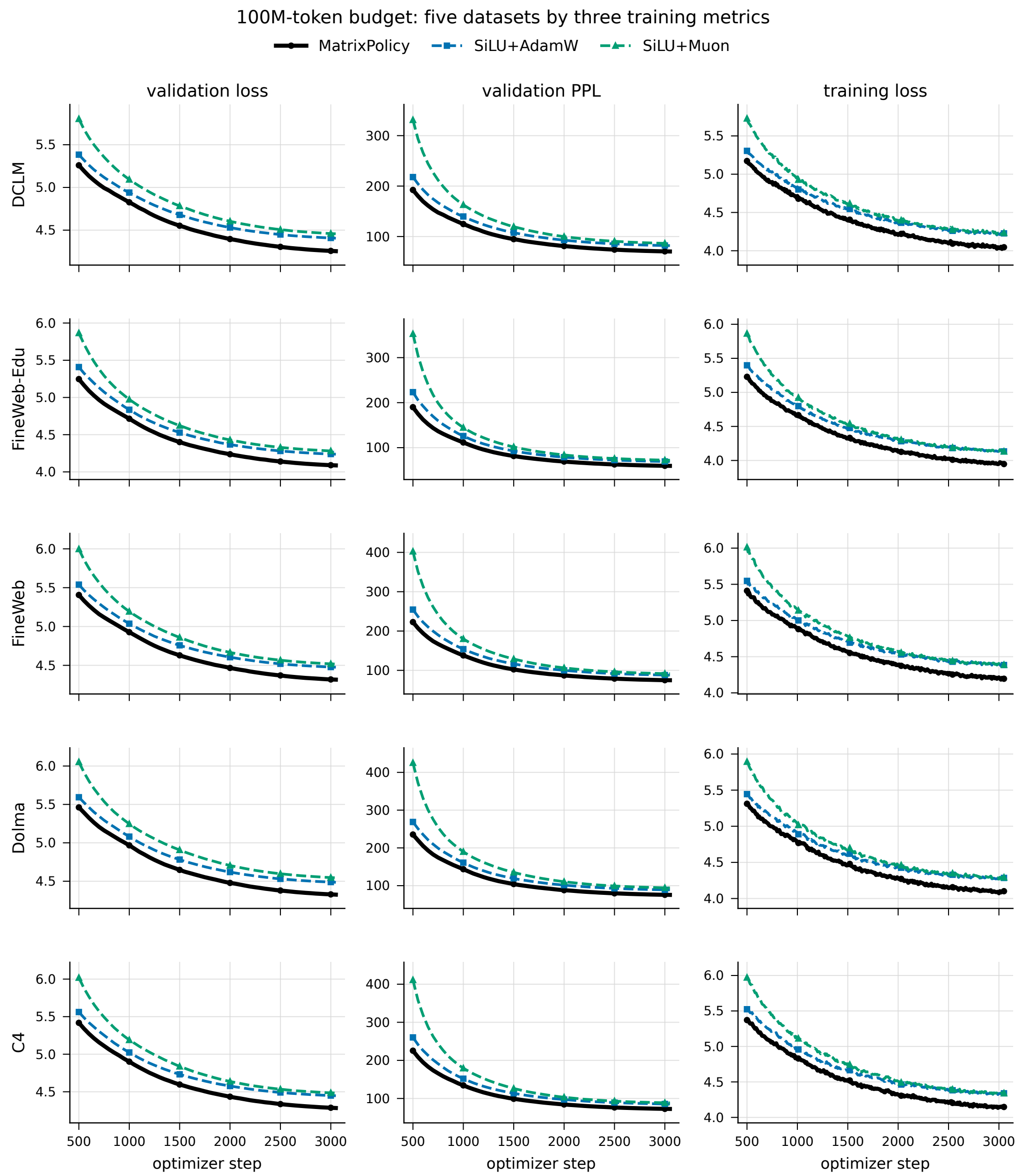

Figure 5: 100M-token-budget dynamics across all five datasets. Rows are datasets and columns are validation loss, validation perplexity, and training loss. Lines are seed means over three runs and bands show one sample standard deviation.

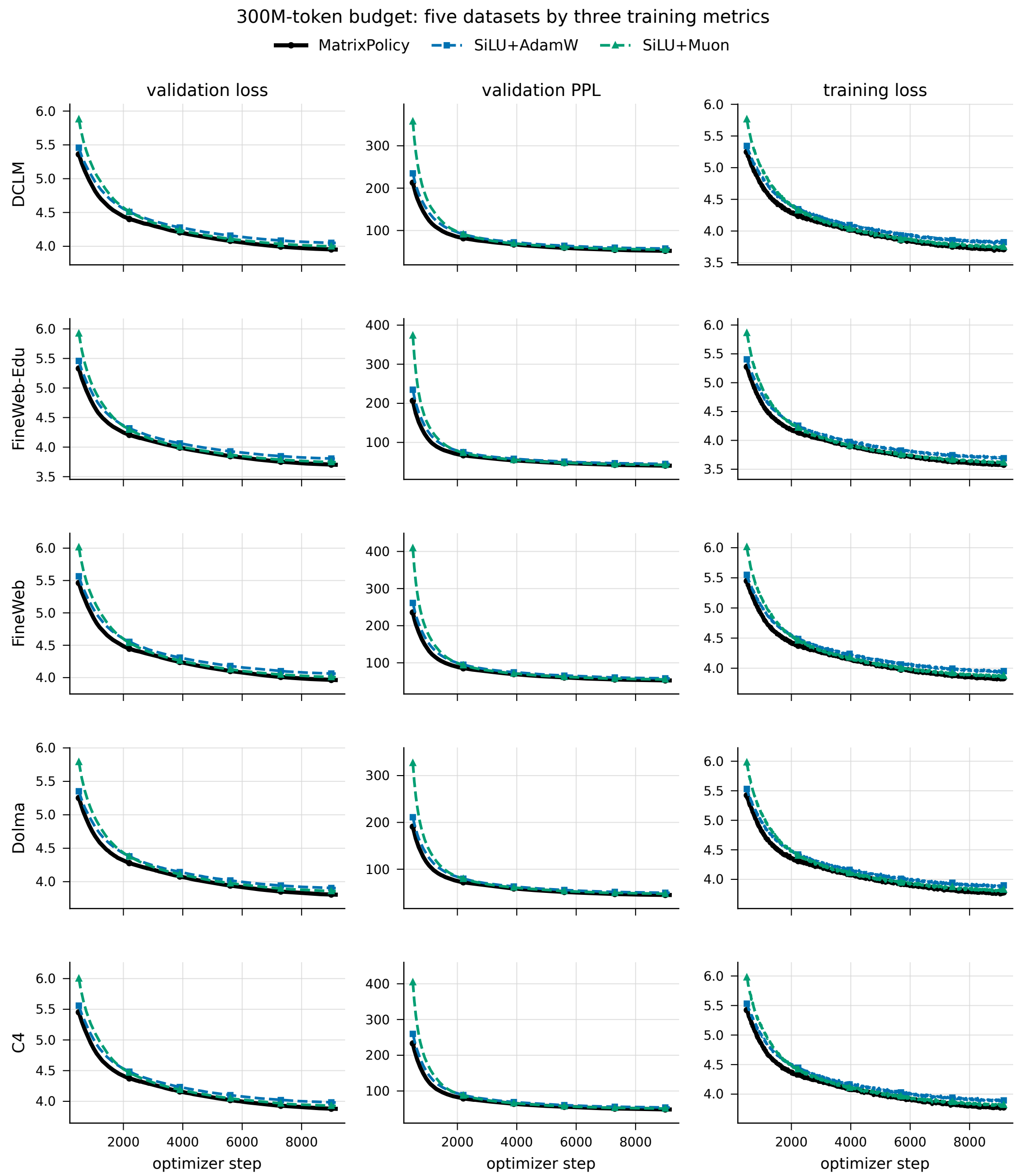

Figure 6: 300M-token-budget dynamics across all five datasets. Rows are datasets and columns are validation loss, validation perplexity, and training loss. Lines are seed means over three runs and bands show one sample standard deviation.

A.1.4 Broad Optimizer Endpoint Table

The fixed-scale package also includes the broader optimizer endpoints referenced from Section 4.1. These rows use the same model, datasets, seeds, token budgets, and validation schedule as the main comparison, while replacing the nonlinear-sublayer optimizer according to the table entry.

Table 2: Final validation loss for the broader optimizer sweep on the fixed 12-layer, width-768 language model. Values are mean $\pm$ sample standard deviation over three seeds; lower is better. RLB rows use the same rational latent-basis activation without MatrixPolicy.

Budget	Method	DCLM	FineWeb-Edu	FineWeb	Dolma	C4
100M tokens	MatrixPolicy	4.254±0.006	4.087±0.010	4.316±0.013	4.325±0.005	4.284±0.020
	SiLU + AdamW	4.406±0.010	4.237±0.009	4.476±0.010	4.486±0.001	4.447±0.016
	SiLU + Lion	4.318±0.007	4.149±0.009	4.383±0.008	4.388±0.005	4.354±0.016
	SiLU + Muon	4.457±0.013	4.279±0.024	4.516±0.026	4.544±0.011	4.482±0.023
	SiLU + SOAP	4.416±0.004	4.263±0.022	4.484±0.011	4.499±0.004	4.458±0.015
	SiLU + ScheduleFree	4.902±0.011	4.826±0.007	5.014±0.019	5.049±0.015	5.006±0.016
	SiLU + CAME	5.011±0.014	4.921±0.012	5.133±0.013	5.165±0.019	5.123±0.018
	RLB + AdamW	4.405±0.008	4.239±0.008	4.472±0.014	4.488±0.003	4.441±0.016
	RLB + Lion	4.295±0.008	4.136±0.008	4.363±0.011	4.362±0.007	4.327±0.016
	RLB + Muon	4.467±0.012	4.283±0.028	4.516±0.006	4.548±0.015	4.476±0.010
	RLB + SOAP	4.436±0.044	4.275±0.024	4.689±0.336	4.518±0.036	4.446±0.019
	RLB + ScheduleFree	4.887±0.005	4.796±0.013	4.999±0.019	5.036±0.013	4.984±0.016
	RLB + CAME	5.014±0.009	4.915±0.006	5.134±0.016	5.151±0.017	5.111±0.016
300M tokens	MatrixPolicy	3.952±0.028	3.702±0.021	3.962±0.008	3.806±0.007	3.878±0.014
	SiLU + AdamW	4.049±0.027	3.803±0.018	4.061±0.010	3.904±0.009	3.981±0.013
	SiLU + Lion	3.993±0.023	3.744±0.021	4.001±0.008	3.848±0.009	3.921±0.011
	SiLU + Muon	3.997±0.030	3.745±0.017	4.007±0.013	3.858±0.010	3.925±0.013
	SiLU + SOAP	4.096±0.030	3.863±0.020	4.114±0.010	3.957±0.009	4.035±0.011
	SiLU + ScheduleFree	4.366±0.030	4.156±0.024	4.398±0.011	4.215±0.005	4.316±0.011
	SiLU + CAME	4.368±0.023	4.150±0.021	4.406±0.019	4.249±0.027	4.330±0.015
	RLB + AdamW	4.050±0.030	3.802±0.021	4.060±0.011	3.905±0.008	3.979±0.010
	RLB + Lion	3.989±0.029	3.742±0.021	3.996±0.011	3.841±0.008	3.913±0.014
	RLB + Muon	3.991±0.026	3.737±0.019	3.999±0.011	3.848±0.005	3.919±0.015
	RLB + SOAP	4.061±0.033	3.824±0.013	4.108±0.032	3.927±0.013	4.002±0.019
	RLB + ScheduleFree	4.361±0.034	4.142±0.022	4.386±0.008	4.212±0.011	4.308±0.007
	RLB + CAME	4.450±0.043	4.225±0.036	4.480±0.006	4.267±0.041	4.365±0.058

A.2 TODO: Larger-Scale Training Results

TODO: Insert larger-scale training details here.

A.3 TODO: Component Ablations

TODO: Insert component ablation details here.

B Rational Latent Basis Calculus

All calculations in this section are local to one layer and one group unless indices are shown explicitly.

B.1 Scalar Response Derivatives

The evaluated response is the groupwise P5/Q4 function in equation 3–equation 5. Writing

$$\phi_1(u) = |u|, \quad \phi_2(u) = u^2, \quad \phi_3(u) = |u|^3, \quad \phi_4(u) = u^4, \qquad Q(u) = 1 + \sum_{j=1}^{4} |b_j|\phi_j(u), \tag{22}$$

gives $Q(u) \geq 1$ on the real line. Thus the scalar response has no real pole, while its numerator and denominator coefficients remain explicit trainable coordinates. Away from $u = 0$,

$$R'(u) = \frac{P'(u)Q(u) - P(u)Q'(u)}{Q(u)^2}, \tag{23}$$

$$P'(u) = \sum_{i=1}^{5} i a_i u^{i-1}, \qquad Q'(u) = |b_1|\operatorname{sign}(u) + 2|b_2|u + 3|b_3|u|u| + 4|b_4|u^3. \tag{24}$$

The derivative-gain summaries use the same autodiff convention as the code, $\operatorname{sign}(0) = 0$ at absolute-value kinks. The coefficient features are

$$\frac{\partial R(u)}{\partial a_i} = \frac{u^i}{Q(u)}, \qquad \frac{\partial R(u)}{\partial b_j} = -\frac{P(u)}{Q(u)^2}\operatorname{sign}(b_j)\phi_j(u), \tag{25}$$

with $\mathrm{sign}(0) = 0$ at $b_j = 0$. Consequently the three summary families in the method have direct scalar origins: $R(u)$ for response gain, $R'(u)$ for derivative gain, and coefficient gradients through the features in equation 25. For the evaluated P5/Q4 response, $\Theta^R_{l,g} = \{a_{l,g,\cdot}, b_{l,g,\cdot}\}$ in equation 12; if another implementation adds further trainable response-coordinate blocks, the same block-average definition extends to those blocks.

B.2 RMS-Normalized Group Map and Positive Pair Coordinate

For a group vector $z \in \mathbb{R}^m$, define

$$r(z) = \sqrt{m^{-1}\|z\|_2^2 + \epsilon_{\mathrm{rms}}}, \qquad u(z) = z/r(z), \qquad h(z) = r(z)R(u(z)), \tag{26}$$

where R acts coordinatewise. On the domain $r(z) > 0$, the radius and normalized-coordinate differentials are

$$\mathrm{d}r = \frac{z^\top \mathrm{d}z}{mr} = \frac{u^\top \mathrm{d}z}{m}, \qquad\qquad \mathrm{d}u = \frac{1}{r}\left(I - \frac{uu^\top}{m}\right)\mathrm{d}z. \tag{27}$$

The factor $I - uu^\top/m$ is the exact differential factor of the implemented normalized coordinate; it becomes the tangent projector to the RMS sphere in the unfloored case. Away from scalar-response kinks, the group Jacobian is

$$J_h(z) := \frac{\partial h}{\partial z} = R(u)\frac{u^\top}{m} + \mathrm{diag}(R'(u))\left(I - \frac{uu^\top}{m}\right). \tag{28}$$

Equation equation 28 separates the radial response term from the normalized-coordinate derivative term. This is why response RMS is an output-role signal and derivative RMS is an input-role signal. They are still proxies: response RMS omits the radius, while derivative RMS omits the radial Jacobian term, the output matrix, and the upstream gradient.

The same normalization gives the positive matrix-pair coordinate used by the wrapper. If $\epsilon_{\mathrm{rms}} = 0$, $z \neq 0$, and $\lambda > 0$, then

$$r(\lambda z) = \lambda r(z), \qquad u(\lambda z) = u(z), \qquad h(\lambda z) = \lambda h(z). \tag{29}$$

Scaling $A_{l,g}$ by λ and the matching block $B_{l,g}$ by λ^{-1} leaves that group contribution unchanged, giving equation 9 groupwise. With the implemented floor, write

$$r_\epsilon(z) = \sqrt{m^{-1}\|z\|_2^2 + \epsilon_{\mathrm{rms}}}, \qquad u_\epsilon(z) = z/r_\epsilon(z), \qquad h_\epsilon(z) = r_\epsilon(z)R(u_\epsilon(z)). \tag{30}$$

For $\rho^2 = m^{-1}\|z\|_2^2$,

$$u_\epsilon(\lambda z) = \kappa(\lambda, z)u_\epsilon(z), \qquad \kappa(\lambda, z) = \frac{\lambda\sqrt{\rho^2 + \epsilon_{\mathrm{rms}}}}{\sqrt{\lambda^2\rho^2 + \epsilon_{\mathrm{rms}}}}. \tag{31}$$

If L_R is a Lipschitz constant of the scalar response on an interval containing the coordinates of $u_\epsilon(z)$ and $\kappa(\lambda, z)u_\epsilon(z)$, then after the reciprocal output compensation,

$$\begin{aligned}\left\|\lambda^{-1}h_\epsilon(\lambda z) - h_\epsilon(z)\right\|_2 \leq{}& \left|\frac{r_\epsilon(\lambda z)}{\lambda} - r_\epsilon(z)\right| \|R(u_\epsilon(z))\|_2 \\ &+ \frac{r_\epsilon(\lambda z)}{\lambda} L_R |\kappa(\lambda, z) - 1| \, \|u_\epsilon(z)\|_2 .\end{aligned} \tag{32}$$

Both terms vanish when $\epsilon_{\mathrm{rms}} = 0$. Under bounded rescaling $|\log\lambda| \leq c$, the deviation is controlled when $e^{2c}\epsilon_{\mathrm{rms}}/\rho^2$ is small and the response is bounded with a bounded derivative on the interval above. Near the RMS floor, the coordinate is therefore a clipped representative-balancing direction rather than an exact symmetry of the implemented floored block.

C MatrixPolicy Update Geometry

C.1 Matrix-Role Gradients and Proxy Status

For one training example, let $\bar{y}_l = \nabla_{y_l}\mathcal{L}$ and $J_{l,g} = \partial h_{l,g}/\partial z_{l,g}$ from equation 28. The adjacent matrix gradients factor as

$$\nabla_{B_{l,g}}\mathcal{L} = \bar{y}_l h_{l,g}^\top, \qquad \nabla_{A_{l,g}}\mathcal{L} = \left(J_{l,g}^\top B_{l,g}^\top \bar{y}_l\right) x_l^\top. \tag{33}$$

The output matrix sees the realized feature, while the input matrix sees the RLB Jacobian before the upstream gradient reaches x_l. The response and derivative RMS summaries in equation 15 are local role proxies because the full gradients also include data, output-matrix, radius, and upstream factors.

The pressure memories in equation 13 summarize post-global-clipping, pre-group-policy gradients as update-magnitude proxies, not signed directional derivatives. Along the exact positive pair direction of the unfloored fixed-curve map, function preservation would cancel the data-loss directional derivative. For nonzero unfloored W and a small update $W^+ = W - \eta\mathcal{G}$ with $\eta\|\mathcal{G}\|_F < c\|W\|_F$ for fixed $c < 1$,

$$\log \operatorname{rms}(W^+) - \log \operatorname{rms}(W) = -\eta \frac{\langle \mathcal{G}, W\rangle_F}{\|W\|_F^2} + O\!\left(\eta^2 \frac{\|\mathcal{G}\|_F^2}{\|W\|_F^2}\right), \tag{34}$$

and Cauchy–Schwarz gives

$$\left|\frac{\langle \mathcal{G}, W\rangle_F}{\|W\|_F^2}\right| \le \frac{\|\mathcal{G}\|_F}{\|W\|_F} = \frac{\operatorname{rms}(\mathcal{G})}{\operatorname{rms}(W)}. \tag{35}$$

The implemented p^{in} and p^{out} replace the last ratio by floored RMS proxies. Floors keep the measurements finite near zero, while the clips in the method turn their log contrasts into bounded controls. Thus $\pi_{l,g}$ identifies which role currently has the larger EMA of the relative update proxy, and $\alpha_{l,g}$ identifies when raw rational-coefficient gradient energy is large relative to adjacent matrix-pressure memories.

C.2 Log-Space Redistribution and Pair-Rescale Dynamics

The group-gradient policy in equation 16 acts in log space. Before clipping, recentering by the group geometric mean gives

$$\widehat{c}_{l,g,\sigma} = \frac{c_{l,g,\sigma}}{\operatorname{geomean}_{j=1}^{G} c_{l,j,\sigma}}, \qquad \frac{1}{G}\sum_{g=1}^{G} \log \widehat{c}_{l,g,\sigma} = 0. \tag{36}$$

The gate redistributes role-specific update scale while preserving the role-wise mean log multiplier before clipping. The pressure signs in equation 16 reduce input-role scale and increase output-role scale when $\pi_{l,g} > 0$.

For pair balancing, ignore floors and clipping temporarily and define

$$\begin{aligned} \gamma_{l,g} &= \log \operatorname{rms}(A_{l,g}) - \log \operatorname{rms}(B_{l,g}), \\ \lambda_{l,g,t} &= s_{l,t}\chi_{l,g}^{\text{rs}}, \\ \gamma_{l,g}^{+} &= \gamma_{l,g} + 2\ell_{l,g,t} = (1 - \lambda_{l,g,t})\gamma_{l,g} + \lambda_{l,g,t}\tau_{l,g}. \end{aligned} \tag{37}$$

The last equality substitutes the unclipped step in equation 21. Because $0 \le s_{l,t} \le 0.625$ and $\chi_{l,g}^{\text{rs}} \le \exp(0.125)$ under the method clips, $0 \le \lambda_{l,g,t} < 1$ before log-step clipping, so the representative coordinate contracts toward $\tau_{l,g}$. This model assumes nonzero matrices, unfloored RMS values, and an unclipped update; the implementation uses floored $\gamma_{l,g}$ and clipped $\ell_{l,g,t}$.

The target in equation 20 equalizes a local representative model that ignores orientation, radius variation, upstream gradients, and floors: input-role scale varies as $\hat{d}^{\text{probe}} e^{-\gamma}$ and output-role scale as $\hat{o}^{\text{probe}} e^{\gamma}$. Equalizing the two log scales gives

$$\gamma_{l,g}^{\star} = \frac{1}{2}\left(\log \hat{d}_{l,g}^{\text{probe}} - \log \hat{o}_{l,g}^{\text{probe}}\right). \tag{38}$$

The implemented target adds the clipped RMS-pressure bias $0.25\bar{\pi}_{l,g} \in [-0.3125, 0.3125]$, while $\chi_{l,g}^{\text{rs}}$ changes only the representative-balancing speed. Together with Appendix B.2, these equations specify the approximation boundary of the pair-rescale move used in the method.